

OPTION B INSTITUTE FAMILY

M07 - POSTGRESQL + DATA MODELING

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

13 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE07-01 - Verify PostgreSQL connection and database/schema context	5
DE07-02 - Tables, data types and explicit DDL	7
DE07-03 - Primary keys, foreign keys and constraints	9
DE07-04 - Transactions, DML and UPSERT	11
DE07-05 - Indexes and EXPLAIN evidence	13
DE07-06 - Operational vs analytical modeling and normalization	15
DE07-07 - Facts, dimensions and star-schema grain	17
DE07-08 - Surrogate keys and referential integrity	19
DE07-09 - Slowly Changing Dimension Type 2 history	21
DE07-10 - Build and validate a small analytical database	23
DE07-11 - Bulk loading with COPY / staging boundaries	25
DE07-12 - Schemas, roles and least-privilege boundaries	27
DE07-13 - Schema change, backup/restore and handoff validation	29
Module mastery checklist + revision quick reference	31

Prerequisite / tool map

Build governed PostgreSQL schemas and analytical models with keys, history and validation.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE07-01 - Verify PostgreSQL connection and database/schema context

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Prove the active database, role and schema search context before changing any object. Teach it from zero A PostgreSQL server (cluster) can contain several databases. A client connection works inside one database at a time, and that database contains schemas. Unqualified table names are resolved through `search_path`. This is why a correct statement against the wrong database or schema is still a serious mistake. M07 reuses the PostgreSQL installation introduced earlier; do not reinstall simply because this module begins.

3. Prerequisite / starting state

M03 PostgreSQL mini-lab completed; M06 terminal basics available.

4. Picture it in your head

VIDEO FIRST route freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners | 0:30:15-0:38:12 - create/connect database + safety orientation Starting state / prerequisite M03 PostgreSQL mini-lab completed; M06 terminal basics available. Outcome Prove the active database, role and schema search context before changing any object.

5. New words before we use them

VIDEO FIRST route freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners | 0:30:15-0:38:12 - create/connect database + safety orientation Starting state / prerequisite M03 PostgreSQL mini-lab completed; M06 terminal basics available. Outcome Prove the active database, role and schema search context before changing any object. Teach it from zero A PostgreSQL server (cluster) can contain several databases.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL Tutorial for Beginners

Assigned scope: 0:00–17:12

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`SELECT current_database() AS db, current_user AS db_user, current_schema() AS current_schema;`
`SHOW search_path;` `SELECT version();` Connect to the dedicated `stage07_de` database. Run the context queries, then create no objects until the database/user/search path evidence is saved.

8. Expected result / what you should see

`current_database()` is `stage07_de`; `current_user` is `known`; `search_path` is recorded; no destructive statement ran before context proof.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If context is wrong, stop. Reconnect to `stage07_de` rather than editing setup SQL to fit the wrong database.

11. If you get stuck

If context is wrong, stop. Reconnect to stage07_de rather than editing setup SQL to fit the wrong database.

12. Small independent task

Close video/book. Reconnect in a fresh query tab or psql session and prove context again. Explain why the GUI tree selection alone is insufficient evidence.

13. Validate

current_database() is stage07_de; current_user is known; search_path is recorded; no destructive statement ran before context proof.

14. Teach it back

Explain verify postgresql connection and database/schema context in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Schemas PostgreSQL 18 - psql Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-02

DE07-02 - Tables, data types and explicit DDL

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL. Teach it from zero Types are part of the data contract. Use date for calendar dates, timestamptz when an instant/time-zone meaning matters, integer/bigint for whole numbers, numeric(p,s) when exact decimal precision matters, boolean for true/false, and text for variable text. DEFAULT supplies a value when a column is omitted; it does not magically replace an explicitly supplied NULL. DDL should live in scripts so a second learner can reproduce the schema.

3. Prerequisite / starting state

DE07-01 mastered.

4. Picture it in your head

VIDEO FIRST route freeCodeCamp - PostgreSQL tables and constraints | 0:41:37-0:55:55 - CREATE TABLE, table structure and constraints Starting state / prerequisite DE07-01 mastered. Outcome Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL.

5. New words before we use them

VIDEO FIRST route freeCodeCamp - PostgreSQL tables and constraints | 0:41:37-0:55:55 - CREATE TABLE, table structure and constraints Starting state / prerequisite DE07-01 mastered. Outcome Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL. Teach it from zero Types are part of the data contract.

6. Useful video / source

Source: freeCodeCamp — detailed PostgreSQL course

Assigned scope: 41:37–55:55

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

CREATE TABLE source.suppliers (supplier_id text PRIMARY KEY, supplier_name text NOT NULL, country text NOT NULL, active boolean NOT NULL DEFAULT true, rating numeric(3,2) CHECK (rating BETWEEN 0 AND 5)); Create source.suppliers from the course specification, then inspect information_schema.columns instead of trusting the GUI alone.

8. Expected result / what you should see

Column definitions match the written contract; dates/numbers are not stored as text merely to avoid parsing; DDL is saved.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a value does not fit the type, decide whether the data is invalid or the type contract was wrong. Do not weaken the type automatically.

11. If you get stuck

If a value does not fit the type, decide whether the data is invalid or the type contract was wrong. Do not weaken the type automatically.

12. Small independent task

Create source.carriers with a text business key, required name, active boolean default, joined_on date and exact rating/score field. Justify each type in comments.

13. Validate

Column definitions match the written contract; dates/numbers are not stored as text merely to avoid parsing; DDL is saved.

14. Teach it back

Explain tables, data types and explicit ddl in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Data Types PostgreSQL 18 - CREATE TABLE Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-03

DE07-03 - Primary keys, foreign keys and constraints

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges. Teach it from zero PRIMARY KEY identifies a row and is unique/non-null. UNIQUE protects alternate keys. FOREIGN KEY requires the referenced parent key to exist. NOT NULL makes presence mandatory. CHECK enforces a row-level predicate. Constraint failures during a controlled test are good evidence: they prove the database boundary is working. Never remove a valid integrity rule simply to make deliberately bad practice data load.

3. Prerequisite / starting state

DE07-02 mastered.

4. Picture it in your head

VIDEO FIRST route freeCodeCamp - PostgreSQL keys and constraints | 2:29:24-2:54:45 + 3:16:41-3:25:04 - primary/unique/check/foreign keys Starting state / prerequisite DE07-02 mastered. Outcome Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges.

5. New words before we use them

VIDEO FIRST route freeCodeCamp - PostgreSQL keys and constraints | 2:29:24-2:54:45 + 3:16:41-3:25:04 - primary/unique/check/foreign keys Starting state / prerequisite DE07-02 mastered. Outcome Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges. Teach it from zero PRIMARY KEY identifies a row and is unique/non-null.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL constraints

Assigned scope: 2:29:24-2:49:15 + 3:16:41-3:25:04

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

INSERT INTO source.suppliers VALUES ('S001','Demo','IN',true,4.50); -- deliberate failures, one at a time: -- duplicate S001; NULL supplier_name; rating 6.00 Run one valid supplier insert, then one duplicate-key, one NULL-required-value and one out-of-range test. Save each error and map it to the constraint.

8. Expected result / what you should see

Each invalid state is rejected for the expected reason; valid rows remain; no constraint was disabled/deleted as a shortcut.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Read the exact violated constraint and identify the business rule. Repair the bad input or model—not the evidence.

11. If you get stuck

Read the exact violated constraint and identify the business rule. Repair the bad input or model—not the evidence.

12. Small independent task

Design a parent carrier table plus a child route table. Prove one orphan route is rejected by the FK and roll back/clean only the test state.

13. Validate

Each invalid state is rejected for the expected reason; valid rows remain; no constraint was disabled/deleted as a shortcut.

14. Teach it back

Explain primary keys, foreign keys and constraints in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Constraints Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-04

DE07-04 - Transactions, DML and UPSERT

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule. Teach it from zero BEGIN starts an explicit transaction, COMMIT makes its changes durable, and ROLLBACK discards uncommitted changes. The strongest rollback proof records before, inside-transaction and after-rollback states. UPSERT is not “update anything that conflicts”; ON CONFLICT must target a meaningful unique key and define which attributes are safe to update. Transactions preserve atomicity boundaries but cannot make incorrect business logic correct.

3. Prerequisite / starting state

DE07-03 mastered.

4. Picture it in your head

VIDEO FIRST route Software Nuggets - PostgreSQL Transactions | 0:40-6:47 - transaction overview, COMMIT and ROLLBACK; then freeCodeCamp 3:01:36-3:16:41 for UPDATE/ON CONFLICT/UPSERT Starting state / prerequisite DE07-03 mastered. Outcome Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule.

5. New words before we use them

VIDEO FIRST route Software Nuggets - PostgreSQL Transactions | 0:40-6:47 - transaction overview, COMMIT and ROLLBACK; then freeCodeCamp 3:01:36-3:16:41 for UPDATE/ON CONFLICT/UPSERT Starting state / prerequisite DE07-03 mastered. Outcome Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule. Teach it from zero BEGIN starts an explicit transaction, COMMIT makes its changes durable, and ROLLBACK discards uncommitted changes.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL DML/transactions

Assigned scope: 2:54:45–3:16:41

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

BEGIN; UPDATE source.customers SET region='South' WHERE customer_id='C001'; SELECT customer_id,region FROM source.customers WHERE customer_id='C001'; ROLLBACK; SELECT customer_id,region FROM source.customers WHERE customer_id='C001'; Run the rollback demonstration, then perform one controlled UPSERT on an existing supplier or dimension row.

8. Expected result / what you should see

Before/inside/after evidence proves rollback; post-rollback source state matches baseline; UPSERT target is a real unique key and only intended fields update.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a GUI autocommits unexpectedly, stop and reproduce in one explicit transaction block. A COMMIT already issued cannot be undone by ROLLBACK.

11. If you get stuck

If a GUI autocommits unexpectedly, stop and reproduce in one explicit transaction block. A COMMIT already issued cannot be undone by ROLLBACK.

12. Small independent task

Change C002 inside a new transaction, prove temporary state, roll back, then write an UPSERT for a different key with a comment explaining conflict policy.

13. Validate

Before/inside/after evidence proves rollback; post-rollback source state matches baseline; UPSERT target is a real unique key and only intended fields update.

14. Teach it back

Explain transactions, dml and upsert in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Transactions tutorial PostgreSQL 18 - INSERT / ON CONFLICT Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-05

DE07-05 - Indexes and EXPLAIN evidence

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data. Teach it from zero An index is a separate structure maintained as table rows change, so it trades storage/write work for potential read efficiency. Index the access pattern, not every column. EXPLAIN shows the planner estimate; EXPLAIN ANALYZE executes and adds actual evidence, so use it carefully. On tiny course tables a sequential scan may remain cheaper even after a sensible index exists. The lab is about evidence, not forcing a plan shape.

3. Prerequisite / starting state

DE07-04 mastered.

4. Picture it in your head

VIDEO FIRST route Milan Jovanovic - How SQL Indexes Actually Work (PostgreSQL) | Whole focused video - B-tree/index choice + EXPLAIN ANALYZE; do not memorize its timing numbers Starting state / prerequisite DE07-04 mastered. Outcome Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data.

5. New words before we use them

VIDEO FIRST route Milan Jovanovic - How SQL Indexes Actually Work (PostgreSQL) | Whole focused video - B-tree/index choice + EXPLAIN ANALYZE; do not memorize its timing numbers Starting state / prerequisite DE07-04 mastered. Outcome Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data. Teach it from zero An index is a separate structure maintained as table rows change, so it trades storage/write work for potential read efficiency.

6. Useful video / source

Source: PostgresConf — Basic understanding of EXPLAIN ANALYZE

Assigned scope: Beginner EXPLAIN/ANALYZE class; use the index-scan / sequential-scan sections

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: Stable PostgreSQL performance concepts; Teaching Book still controls the exact evidence task.

[Open source](#)

7. Follow with me once

EXPLAIN SELECT * FROM source.orders WHERE customer_id='C003'; CREATE INDEX IF NOT EXISTS idx_orders_customer_id ON source.orders(customer_id); EXPLAIN SELECT * FROM source.orders WHERE customer_id='C003'; Capture before/after plans for the customer lookup and write only what actually changed. Record that Seq Scan on tiny data is acceptable.

8. Expected result / what you should see

Claim is supported by observed plan evidence; no “index always faster” statement; index matches a real predicate/join/order pattern.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the planner still uses Seq Scan, do not force an index. Explain table size/selectivity and keep the evidence.

11. If you get stuck

If the planner still uses Seq Scan, do not force an index. Explain table size/selectivity and keep the evidence.

12. Small independent task

Choose a different repeated filter/join from the lab, propose one index, capture before/after EXPLAIN and state one write/storage cost.

13. Validate

Claim is supported by observed plan evidence; no “index always faster” statement; index matches a real predicate/join/order pattern.

14. Teach it back

Explain indexes and explain evidence in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Indexes PostgreSQL 18 - Using EXPLAIN Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-06

DE07-06 - Operational vs analytical modeling and normalization

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation. Teach it from zero. Operational models usually separate business entities so one fact or attribute has an authoritative home and updates do not require changing dozens of copied values. Analytical models often reorganize data around measurable events and descriptive dimensions. Normalization is not a moral score: the right boundary follows workload, ownership, integrity and grain. Before deciding table boundaries, state what one row means and which business key identifies it.

3. Prerequisite / starting state

DE07-05 mastered.

4. Picture it in your head

VIDEO FIRST route Maven Analytics - Database Normalization for Beginners | 0:13-1:56 normalization meaning + 10:57-18:00 2NF/3NF/pros-cons; use 1:56-9:36 only as relational/star visual support
Starting state / prerequisite DE07-05 mastered. Outcome Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation.

5. New words before we use them

VIDEO FIRST route Maven Analytics - Database Normalization for Beginners | 0:13-1:56 normalization meaning + 10:57-18:00 2NF/3NF/pros-cons; use 1:56-9:36 only as relational/star visual support
Starting state / prerequisite DE07-05 mastered. Outcome Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation. Teach it from zero. Operational models usually separate business entities so one fact or attribute has an authoritative home and updates do not require changing dozens of copied values.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Main modeling class — operational vs analytical thinking + dimensional-model context

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: Use as the visual teacher; course lab supplies the exact PostgreSQL implementation.

[Open source](#)

7. Follow with me once

Operational: customers -> orders -> order_items. Customer name/region live once in customers.

Analytical: fact_sales repeats di Inspect source.customers/orders/order_items and list three update/anomaly problems that would appear if customer_name and region were copied into every order line.

8. Expected result / what you should see

You distinguish harmful duplicated attributes from intentional repeated foreign keys at fact grain; every proposed table has a written grain.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you are arguing “normalized good / denormalized bad,” return to workload + integrity + grain and explain the trade-off.

11. If you get stuck

If you are arguing “normalized good / denormalized bad,” return to workload + integrity + grain and explain the trade-off.

12. Small independent task

Take a wide carrier-delivery row and sketch an operational decomposition. Then state which repeated keys would be acceptable in an analytical fact.

13. Validate

You distinguish harmful duplicated attributes from intentional repeated foreign keys at fact grain; every proposed table has a written grain.

14. Teach it back

Explain operational vs analytical modeling and normalization in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - DDL basics Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-07

DE07-07 - Facts, dimensions and star-schema grain

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals. Teach it from zero A fact table records measurable events at a declared grain and stores dimension foreign keys. Dimensions hold descriptive context used to group/filter. In this lab warehouse.fact_sales is one row per COMPLETED order line—not one row per order. One order can legitimately have several fact rows. Aggregating by region changes result grain to one row per region, but the regional values must reconcile to the overall fact measure.

3. Prerequisite / starting state

DE07-06 mastered.

4. Picture it in your head

VIDEO FIRST route Kahan Data Solutions - Data Modeling Tutorial: Star Schema | 0:31-14:55 - overview, facts, dimensions, marts and benefits Starting state / prerequisite DE07-06 mastered. Outcome Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals.

5. New words before we use them

VIDEO FIRST route Kahan Data Solutions - Data Modeling Tutorial: Star Schema | 0:31-14:55 - overview, facts, dimensions, marts and benefits Starting state / prerequisite DE07-06 mastered. Outcome Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals. Teach it from zero A fact table records measurable events at a declared grain and stores dimension foreign keys.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Main modeling class — facts, dimensions, grain and star-schema reasoning

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: One coherent class may support several lessons. Do not rewatch from zero if the relevant idea is already clear.

[Open source](#)

7. Follow with me once

SELECT c.region, SUM(f.gross_sales) AS revenue FROM warehouse.fact_sales f JOIN warehouse.dim_customer c ON c.customer_key=f.customer_key GROUP BY c.region ORDER BY c.region; Write the fact grain sentence before querying. Produce revenue by region and product category; reconcile both grouped totals to overall fact revenue.

8. Expected result / what you should see

fact_sales grain is correct; 12 rows, qty 30, revenue 14130.00; region/category grouped results each sum back to 14130.00.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If totals inflate, inspect join cardinality and measure grain before using DISTINCT. DISTINCT is not a repair for unexplained fan-out.

11. If you get stuck

If totals inflate, inspect join cardinality and measure grain before using DISTINCT. DISTINCT is not a repair for unexplained fan-out.

12. Small independent task

Design a delivery fact at one row per delivered parcel line with date/customer/product-like dimensions. Identify additive measures and one measure that could be duplicated at the wrong grain.

13. Validate

fact_sales grain is correct; 12 rows, qty 30, revenue 14130.00; region/category grouped results each sum back to 14130.00.

14. Teach it back

Explain facts, dimensions and star-schema grain in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) Microsoft - Understand star schema Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-08

DE07-08 - Surrogate keys and referential integrity

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references. Teach it from zero A business key such as `customer_id` comes from the source/business. A surrogate key is generated inside the warehouse and has no business meaning by itself. The dimension should keep the business key for traceability while facts use the surrogate key to reference the exact warehouse row. This becomes especially valuable when one business key can have multiple historical dimension versions. Referential integrity must be proven, not assumed.

3. Prerequisite / starting state

DE07-07 mastered.

4. Picture it in your head

VIDEO FIRST route EduFulness - Slowly Changing Dimensions Masterclass | 8:00-24:28 - keys, surrogate key, fact and dimension foundations Starting state / prerequisite DE07-07 mastered. Outcome Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references.

5. New words before we use them

VIDEO FIRST route EduFulness - Slowly Changing Dimensions Masterclass | 8:00-24:28 - keys, surrogate key, fact and dimension foundations Starting state / prerequisite DE07-07 mastered. Outcome Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references. Teach it from zero A business key such as `customer_id` comes from the source/business.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Main modeling class — dimensional model keys/grain; Teaching Book adds surrogate-key rules

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: Video gives the model; the book/lab handles the exact referential-integrity implementation.

[Open source](#)

7. Follow with me once

`SELECT customer_key, customer_id, customer_name, region FROM warehouse.dim_customer ORDER BY customer_key;` -- orphan checks should return 0 Display business/surrogate key pairs, then run explicit orphan checks for `customer_key` and `product_key`.

8. Expected result / what you should see

orphan customer/product refs = 0; invalid FK insert is rejected; natural key remains visible in dimension; surrogate key is not treated as user-facing meaning.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If an orphan exists, trace source key -> dimension lookup/load -> fact load. Never replace unknown keys with arbitrary existing surrogate keys.

11. If you get stuck

If an orphan exists, trace source key -> dimension lookup/load -> fact load. Never replace unknown keys with arbitrary existing surrogate keys.

12. Small independent task

Inside a transaction, attempt a fact row with a nonexistent product_key, save the FK error, then roll back. Explain why storing product_name as the relationship key would be weaker.

13. Validate

orphan customer/product refs = 0; invalid FK insert is rejected; natural key remains visible in dimension; surrogate key is not treated as user-facing meaning.

14. Teach it back

Explain surrogate keys and referential integrity in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Identity columns PostgreSQL 18 - Foreign keys Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-09

DE07-09 - Slowly Changing Dimension Type 2 history

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Preserve attribute history with effective dates/current flags instead of overwriting the past. Teach it from zero SCD Type 2 retains multiple versions of a dimension business key. A current row is closed and a new row is inserted when a tracked attribute changes. Typical fields are `effective_from`, `effective_to` and `is_current`. A transaction keeps the close-old/open-new steps together. The toy lab deliberately teaches the pattern but does not pretend one CHECK constraint solves all production overlap/concurrency cases.

3. Prerequisite / starting state

DE07-08 mastered.

4. Picture it in your head

VIDEO FIRST route EduFulness - Slowly Changing Dimensions Masterclass | 36:48-42:06 SCD why/what; 1:09:05-1:40:54 SCD Type 2 Starting state / prerequisite DE07-08 mastered. Outcome Preserve attribute history with effective dates/current flags instead of overwriting the past.

5. New words before we use them

VIDEO FIRST route EduFulness - Slowly Changing Dimensions Masterclass | 36:48-42:06 SCD why/what; 1:09:05-1:40:54 SCD Type 2 Starting state / prerequisite DE07-08 mastered. Outcome Preserve attribute history with effective dates/current flags instead of overwriting the past. Teach it from zero SCD Type 2 retains multiple versions of a dimension business key.

6. Useful video / source

Source: TechLake — Slowly Changing Dimensions and Types

Assigned scope: Whole focused SCD class; concentrate on Type 1 vs Type 2 history

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: Older recording is acceptable here because SCD Type 1/2 concepts are stable; implementation is validated in our current PostgreSQL lab.

[Open source](#)

7. Follow with me once

```
-- C003 moves North -> East effective 2026-09-01 BEGIN; UPDATE warehouse.dim_customer_scd2 SET
effective_to='2026-08-31',is_current=false WHERE customer_id='C003' AND is_current=true; -- insert
new current C003 East version COMMIT; Run the SCD2 lab. Prove C003 has two versions and exactly
one is_current=true, then narrate what history would be lost by an in-place UPDATE.
```

8. Expected result / what you should see

Old version remains; new version starts on intended date; exactly one current version for changed key; effective dates do not overlap in the taught example.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If two current rows remain, stop and repair the close-old/insert-new transaction. Do not delete the old history row.

11. If you get stuck

If two current rows remain, stop and repair the close-old/insert-new transaction. Do not delete the old history row.

12. Small independent task

Apply a changed future segment update to a different customer in a fresh SCD2 copy. Save before/after history and check exactly one current row.

13. Validate

Old version remains; new version starts on intended date; exactly one current version for changed key; effective dates do not overlap in the taught example.

14. Teach it back

Explain slowly changing dimension type 2 history in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) Microsoft Fabric - Slowly changing dimension Type 2 Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-10

DE07-10 - Build and validate a small analytical database

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation. Teach it from zero A successful CREATE/INSERT run is not enough. A reliable build has objective invariants: declared grain, row counts, eligibility/exclusion rules, unique business grain, zero orphans and monetary/quantity totals that reconcile. The setup is intentionally rerunnable at the module level; validation must be rerun after any material model change.

3. Prerequisite / starting state

DE07-01 through DE07-09 mastered.

4. Picture it in your head

VIDEO FIRST route Kahan Data Solutions - Data Modeling Tutorial: Star Schema | 0:31-14:55 - quick model-design refresh only; do NOT copy a project solution Starting state / prerequisite DE07-01 through DE07-09 mastered. Outcome Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation.

5. New words before we use them

VIDEO FIRST route Kahan Data Solutions - Data Modeling Tutorial: Star Schema | 0:31-14:55 - quick model-design refresh only; do NOT copy a project solution Starting state / prerequisite DE07-01 through DE07-09 mastered. Outcome Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation. Teach it from zero A successful CREATE/INSERT run is not enough.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling
 Assigned scope: Revisit only the model-design parts needed before building the analytical database
 How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.
 Note: Build remains independent; this is not a copy-the-project solution.
[Open source](#)

7. Follow with me once

SELECT COUNT(*) AS fact_rows, SUM(qty) AS total_qty, SUM(gross_sales) AS total_revenue FROM warehouse.fact_sales; -- Expected baseline: 12, 30, 14130.00 Run the canonical setup/build then final validation. Explain every control: 12 fact rows, qty 30, revenue 14130.00, cancelled O1005 contributes zero, no duplicate (order_id,line_no), zero orphans.

8. Expected result / what you should see

All baseline controls pass and region totals East 3710, North 2390, South 1750, West 6280 sum to 14130.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If one control fails, isolate eligibility -> dimension lookup -> fact grain -> measure calculation. Do not adjust expected totals to match a broken load.

11. If you get stuck

If one control fails, isolate eligibility -> dimension lookup -> fact grain -> measure calculation. Do not adjust expected totals to match a broken load.

12. Small independent task

From a clean stage07_de rebuild, execute the build and validation without looking at the teacher SQL except when repairing. Save a validation transcript.

13. Validate

All baseline controls pass and region totals East 3710, North 2390, South 1750, West 6280 sum to 14130.

14. Teach it back

Explain build and validate a small analytical database in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 documentation Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-11

DE07-11 - Bulk loading with COPY / staging boundaries

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy. Teach it from zero COPY is a PostgreSQL SQL command; when it names a file, the server process reads/writes from the server environment and is constrained by server-side permissions. psql \copy is a psql meta-command: the client reads/writes the local file and streams data through the connection. In a learner workstation this distinction explains why a local path can work with \copy but fail with server-side COPY. Staging is a receiving area: load permissively enough to inspect, reconcile source business rows, then transform into final constrained tables.

3. Prerequisite / starting state

DE07-10 mastered.

4. Picture it in your head

VIDEO FIRST route vlogize - Importing CSV Files into PostgreSQL Tables | Whole focused video - COPY vs client-side \copy/file-location problem; Teaching Book controls exact course method Starting state / prerequisite DE07-10 mastered. Outcome Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy.

5. New words before we use them

VIDEO FIRST route vlogize - Importing CSV Files into PostgreSQL Tables | Whole focused video - COPY vs client-side \copy/file-location problem; Teaching Book controls exact course method Starting state / prerequisite DE07-10 mastered. Outcome Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy. Teach it from zero COPY is a PostgreSQL SQL command; when it names a file, the server process reads/writes from the server environment and is constrained by server-side permissions.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use COPY / loading section if present; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
CREATE TABLE source.stg_customers(customer_id text,customer_name text,region text,segment text);
-- psql client-side: -- \copy source.stg_customers FROM 'customers_stage.csv' WITH (FORMAT csv,
HEADER true); SELECT COUNT(*) FROM source.stg_customers; Load the bundled customers_stage.csv
into source.stg_customers using psql \copy or documented pgAdmin Import/Export. Save source
business-row count and staging COUNT(*)
```

8. Expected result / what you should see

Source and staging business-row counts reconcile; header is not counted; learner explains COPY vs \copy and staging vs final tables.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a local path fails with COPY, do not change file permissions blindly. First determine whether the command is server-side COPY or client-side \copy.

11. If you get stuck

If a local path fails with COPY, do not change file permissions blindly. First determine whether the command is server-side COPY or client-side \copy.

12. Small independent task

Load carriers_stage.csv to a new staging table, reconcile rows, flag one malformed/unknown region row before any final constrained insert.

13. Validate

Source and staging business-row counts reconcile; header is not counted; learner explains COPY vs \copy and staging vs final tables.

14. Teach it back

Explain bulk loading with copy / staging boundaries in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - COPY PostgreSQL 18 - psql \copy Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-12

DE07-12 - Schemas, roles and least-privilege boundaries

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Design schema/role boundaries that grant only the operations a workload needs. Teach it from zero PostgreSQL roles can be login identities or group-like privilege containers. Database CONNECT, schema USAGE/CREATE and table SELECT/INSERT/UPDATE/DELETE are different powers. Object ownership is stronger than ordinary grants. A reporting workload normally should not run as the database owner or superuser. Least privilege means writing down what the workload needs and explicitly what it does not need. Default privileges/future objects are a separate consideration from existing tables.

3. Prerequisite / starting state

DE07-11 mastered.

4. Picture it in your head

VIDEO FIRST route Microsoft Developer - Roles, Privileges, and Security in PostgreSQL | 13:56-21:57 privileges + principle of least privilege; optional 21:57 onward demo Starting state / prerequisite DE07-11 mastered. Outcome Design schema/role boundaries that grant only the operations a workload needs.

5. New words before we use them

VIDEO FIRST route Microsoft Developer - Roles, Privileges, and Security in PostgreSQL | 13:56-21:57 privileges + principle of least privilege; optional 21:57 onward demo Starting state / prerequisite DE07-11 mastered. Outcome Design schema/role boundaries that grant only the operations a workload needs. Teach it from zero PostgreSQL roles can be login identities or group-like privilege containers.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use roles/privileges section if present; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

CREATE ROLE stage07_reader NOLOGIN; GRANT USAGE ON SCHEMA warehouse TO stage07_reader; GRANT SELECT ON ALL TABLES IN SCHEMA warehouse TO stage07_reader; Design warehouse_reader and source_loader privilege matrices. If your local account is allowed, create only a NOLOGIN reader and inspect grants; otherwise do the design evidence without changing system privileges.

8. Expected result / what you should see

Reader has no write/drop/ownership requirement; loader does not receive warehouse-owner powers; design distinguishes schema USAGE from table privileges.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If permission errors appear, inspect `current_user`, ownership and actual grants. Do not switch the application to postgres/superuser as the repair.

11. If you get stuck

If permission errors appear, inspect `current_user`, ownership and actual grants. Do not switch the application to postgres/superuser as the repair.

12. Small independent task

For a fresh reporting service, state exact database/schema/table privileges, owner boundary and how future-table access would be handled. Reject GRANT ALL as an unexplained shortcut.

13. Validate

Reader has no write/drop/ownership requirement; loader does not receive warehouse-owner powers; design distinguishes schema USAGE from table privileges.

14. Teach it back

Explain schemas, roles and least-privilege boundaries in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - Privileges PostgreSQL 18 - Schemas and privileges Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

M07 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-13

DE07-13 - Schema change, backup/restore and handoff validation

1. Why it matters

This matters because later PostgreSQL + Data Modeling work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy. Teach it from zero Schema change should be scripted, reviewed and validated rather than hidden in GUI clicks. `pg_dump` creates a logical export; custom format (-Fc) is restored with `pg_restore` and supports selective restore features. A backup file is not recovery proof until a restore to a separate database/environment succeeds and the same controls pass. Never place a real password in the command text or evidence. A handoff must include build/migration scripts, grains, control totals, role assumptions, backup metadata, restore validation and known limitations.

3. Prerequisite / starting state

DE07-12 mastered.

4. Picture it in your head

VIDEO FIRST route WittCode - Backing Up a Postgres Database with `pg_dump` and `pg_restore` | 0:34-6:06 - `pg_dump`, formats, `pg_restore` and restore demonstration Starting state / prerequisite DE07-12 mastered. Outcome Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy.

5. New words before we use them

VIDEO FIRST route WittCode - Backing Up a Postgres Database with `pg_dump` and `pg_restore` | 0:34-6:06 - `pg_dump`, formats, `pg_restore` and restore demonstration Starting state / prerequisite DE07-12 mastered. Outcome Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy. Teach it from zero Schema change should be scripted, reviewed and validated rather than hidden in GUI clicks.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use backup/restore/schema-change support only; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`ALTER TABLE source.products ADD COLUMN IF NOT EXISTS active boolean NOT NULL DEFAULT true; --`
terminal examples: `-- pg_dump -Fc -d stage07_de -f stage07_de.backup -- createdb stage07_restore_test -- pg_restore -d stage07_restore_test stage07_de.backup` Apply the active-column migration and prove its definition. If `pg_dump/pg_restore` are available, dump and restore to `stage07_restore_test`, then rerun final controls.

8. Expected result / what you should see

Migration is scripted; backup metadata recorded; restore target is separate; post-restore controls match baseline where runtime is available. If runtime is unavailable, limitation is recorded—not guessed.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If restore fails, keep original database untouched, inspect dump format/version/target/permissions, and retry only against the separate test target.

11. If you get stuck

If restore fails, keep original database untouched, inspect dump format/version/target/permissions, and retry only against the separate test target.

12. Small independent task

Write a handoff checklist and a changed reversible migration for a new nullable source attribute. Include rollback/recovery thought, validation and restore target separation.

13. Validate

Migration is scripted; backup metadata recorded; restore target is separate; post-restore controls match baseline where runtime is available. If runtime is unavailable, limitation is recorded—not guessed.

14. Teach it back

Explain schema change, backup/restore and handoff validation in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success. Current authoritative reference(s) PostgreSQL 18 - pg_dump PostgreSQL 18 - pg_restore Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.